

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №1
по дисциплине
«Современные системы программирования»

Выполнила:

ст. гр. 320604
К. А. Хаджинова

Проверила:

Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	4
3 ВЫПОЛНЕНИЕ РАБОТЫ	5
3.1 Подготовка среды разработки	5
3.2 Анализ <i>IL</i> -кода	5
3.3 Задание 1. Анализ <i>IL</i> -кода базовых конструкций	6
3.4 Генерация и выполнение <i>IL</i> -кода	7
3.5 Задание 2. Работа с <i>IL</i> -генератором	8
3.6 Задание 3. Создание анализатора кода	10
3.7 Исследование <i>JIT</i> -компиляции	11
3.8 Задание 4.1. Влияние <i>JIT</i> -компиляции на сложение элементов массива	13
4 КОНТРОЛЬНЫЕ ВОПРОСЫ	15
4.1 Что такое <i>IL</i> -код и зачем он нужен	15
4.2 Разница между <i>Normal JIT</i> и <i>Tiered Compilation</i>	15
4.3 Холодный старт и способы его уменьшения	15
4.4 Оптимизация часто вызываемых методов	15
4.5 Преимущества генерации кода через <i>ILGenerator</i>	15
ЗАКЛЮЧЕНИЕ	17

1 ЦЕЛЬ РАБОТЫ

Изучить структуру и принципы работы *Intermediate Language (IL)*, исследовать влияние *JIT*-компиляции на производительность.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

IL (*Intermediate Language*) – промежуточный язык, на который компилируется весь код экосистемы *.NET* (*C#*, *F#* и другие). Язык имеет стековую архитектуру: операции берут операнды с вершины стека вычислений и возвращают результат в тот же стек. Объектная модель *IL* поддерживает классы, интерфейсы и наследование.

JIT-компиляция (*Just-In-Time*) – компиляция *IL*-кода в машинный код в процессе выполнения программы. Режим *Normal JIT* компилирует метод при первом вызове. Режим *Tiered Compilation* адаптивный: метод сначала быстро компилируется без глубоких оптимизаций, а часто вызываемые методы затем перекомпилируются с оптимизацией.

Класс *DynamicMethod* вместе с *ILGenerator* позволяет создавать методы во время выполнения без исходного кода на *C#*. Для измерения скорости и выделяемой памяти применяется библиотека *BenchmarkDotNet*.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Подготовка среды разработки

Работа выполнена в консольном приложении *C#* на платформе *.NET*. Проект *Lab1* собран утилитой *dotnet* (SDK 9.0.318) с целевой платформой *net8.0*, как допускает методика. Библиотека *BenchmarkDotNet* версии 0.14.0 подключена пакетом *NuGet*. Файл проекта приведен в листинге.

Листинг 3.1 – Файл проекта *Lab1.csproj*

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<PropertyGroup>
```

```
<OutputType>Exe</OutputType>
```

```
<TargetFramework>net8.0</TargetFramework>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<Nullable>enable</Nullable>
```

```
<AllowUnsafeBlocks>true</AllowUnsafeBlocks>
```

```
</PropertyGroup>
```

```
<ItemGroup>
```

```
<PackageReference Include="BenchmarkDotNet" Version="0.14.0" />
```

```
</ItemGroup>
```

```
</Project>
```

Для запуска заданий 1, 2 и 4 используется команда *dotnet run*, для задания 3 – команда *dotnet run -c Release -- bench*.

3.2 Анализ *IL*-кода

Программа из методических указаний получает тело метода через *GetMethodBody()* и выводит размер *IL*, число локальных переменных, размер стека и байты *IL* в шестнадцатеричном виде. Анализируются методы *SimpleMethod*, *MethodWithLoop* и *MethodWithCondition*. Результат показан на рисунке 3.1.

```

> dotnet run
=== Лабораторная работа: Анализ IL-кода ===

--- Анализ: Метод SimpleMethod ---
Размер IL: 4 байт
Локальные переменные: 0
Максимальный размер стека: 8

IL-инструкции (HEX):
02 03 58 2A

--- Анализ: Метод MethodWithLoop ---
Размер IL: 32 байт
Локальные переменные: 4
Максимальный размер стека: 2

IL-инструкции (HEX):
00 16 0A 17 0B 2B 08 06 07 58 0A 07 17 58 0B 07
02 FE 02 16 FE 01 0C 08 2D ED 06 0D 2B 00 09 2A

--- Анализ: Метод MethodWithCondition ---
Размер IL: 43 байт
Локальные переменные: 3
Максимальный размер стека: 2

IL-инструкции (HEX):
00 02 16 FE 02 0A 06 2C 08 72 91 01 00 70 0B 2B
18 02 16 FE 04 0C 08 2C 08 72 A3 01 00 70 0B 2B
08 72 B5 01 00 70 0B 2B 00 07 2A

```

Рисунок 3.1 – Анализ *IL*-кода методов примера

Метод *SimpleMethod* занимает 4 байта: 02 03 58 2A соответствует командам *ldarg.0*, *ldarg.1*, *add* и *ret*. Метод с циклом занимает 32 байта и использует четыре локальные переменные, метод с условием – 43 байта и три переменные. Значение максимального стека 8 для коротких методов выдается из-за особенностей компактного заголовка тела метода.

3.3 Задание 1. Анализ *IL*-кода базовых конструкций

Для задания написан метод нахождения произведения двух целых чисел. Его *IL*-код получен тем же методом *AnalyzeIL*. Исходный код метода приведен в листинге.

Листинг 3.2 – Метод *Multiply*
 // Задание 1: произведение двух целых чисел
 static int Multiply(int a, int b) => a * b;

Результат работы программы и *IL*-код метода показаны на рисунке 3.2.

```

> dotnet run
=== Задание 1: произведение двух целых чисел ===
Multiply(6, 7) = 42

--- Анализ: Метод Multiply ---
Размер IL: 4 байт
Локальные переменные: 0
Максимальный размер стека: 8

IL-инструкции (HEX):
02 03 5A 2A

```

Рисунок 3.2 – IL-код метода *Multiply*

Метод *Multiply(6, 7)* возвращает 42. Последовательность 02 03 5A 2A читается так: *ldarg.0* помещает в стек первый аргумент, *ldarg.1* – второй, *mul* извлекает два значения и помещает их произведение, *ret* возвращает значение с вершины стека. Отличие от метода сложения только в коде операции: 5A вместо 58.

3.4 Генерация и выполнение IL-кода

Методические указания содержат класс *ILGeneratorDemo* с тремя динамическими методами: сложение двух чисел, факториал через цикл и определение знака числа. Методы создаются через *DynamicMethod*, команды записываются вызовами *il.Emit*, а результат преобразуется в делегат. Результат вызова методов показан на рисунке 3.3.

```

> dotnet run
=== Динамическая генерация IL ===

DynamicAdd(15, 27) = 42
DynamicFactorial(5) = 120
DynamicFactorial(7) = 5040
DynamicGrade(10) = Positive
DynamicGrade(-5) = Negative
DynamicGrade(0) = Zero

```

Рисунок 3.3 – Результат вызова динамически созданных методов

Динамические методы возвращают те же значения, что и обычный код: сумма 15 и 27 равна 42, факториалы 5 и 7 равны 120 и 5040, знак чисел 10, минус 5 и 0 определяется как *Positive*, *Negative* и *Zero*.

3.5 Задание 2. Работа с *IL*-генератором

В задании 2.1 динамический метод вычисляет $n!$ рекурсивно. Метод сравнивает n с единицей, при n не больше 1 возвращает 1, иначе умножает n на результат рекурсивного вызова самого себя командой *call*. В задании 2.2 метод находит максимум из трех чисел: значение первого аргумента сохраняется в локальной переменной, затем по очереди сравнивается со вторым и третьим. Код генерации приведен в листингах.

Листинг 3.3 – Рекурсивный факториал (задание 2.1)

```
// Задание 2.1:  $n!$  рекурсивно
// int Factorial(int n) { if (n <= 1) return 1; return n * Factorial(n - 1); }
public static Func<int, int> CreateRecursiveFactorialMethod()
{
    DynamicMethod dynamicMethod = new DynamicMethod(
        "RecursiveFactorial",
        typeof(int),
        new[] { typeof(int) },
        typeof(ILGeneratorDemo).Module
    );

    ILGenerator il = dynamicMethod.GetILGenerator();

    Label recurse = il.DefineLabel();

    // if (n > 1) goto recurse;
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Bgt, recurse);

    // return 1;
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Ret);

    // recurse: return n * Factorial(n - 1);
    il.MarkLabel(recurse);
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldc_I4_1);
    il.Emit(OpCodes.Sub);
    il.Emit(OpCodes.Call, dynamicMethod); // рекурсивный вызов самого метода
    il.Emit(OpCodes.Mul);
    il.Emit(OpCodes.Ret);

    return (Func<int, int>)dynamicMethod.CreateDelegate(typeof(Func<int, int>));
}
```

Листинг 3.4 – Максимум из трех чисел (задание 2.2)

```
// Задание 2.2: максимум из трёх чисел
```

```

// int Max(int a, int b, int c) { int max = a; if (b > max) max = b; if (c > max) max = c; return
max; }
public static Func<int, int, int, int> CreateMaxOfThreeMethod()
{
    DynamicMethod dynamicMethod = new DynamicMethod(
        "DynamicMax",
        typeof(int),
        new[] { typeof(int), typeof(int), typeof(int) },
        typeof(ILGeneratorDemo).Module
    );

    ILGenerator il = dynamicMethod.GetILGenerator();

    Label skipB = il.DefineLabel();
    Label skipC = il.DefineLabel();

    il.DeclareLocal(typeof(int)); // max

    // max = a;
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Stloc_0);

    // if (b <= max) goto skipB; max = b;
    il.Emit(OpCodes.Ldarg_1);
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ble, skipB);
    il.Emit(OpCodes.Ldarg_1);
    il.Emit(OpCodes.Stloc_0);
    il.MarkLabel(skipB);

    // if (c <= max) goto skipC; max = c;
    il.Emit(OpCodes.Ldarg_2);
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ble, skipC);
    il.Emit(OpCodes.Ldarg_2);
    il.Emit(OpCodes.Stloc_0);
    il.MarkLabel(skipC);

    // return max;
    il.Emit(OpCodes.Ldloc_0);
    il.Emit(OpCodes.Ret);

    return (Func<int, int, int, int>)dynamicMethod.CreateDelegate(typeof(Func<int, int, int,
int>));
}

```

Результаты вызова сгенерированных методов показаны на рисунке 3.4.

```

> dotnet run
=== Задание 2.1: n! рекурсивно ===

RecursiveFactorial(5) = 120
RecursiveFactorial(7) = 5040

=== Задание 2.2: максимум из трёх чисел ===

DynamicMax(3, 9, 5) = 9
DynamicMax(10, 2, 7) = 10
DynamicMax(1, 4, 8) = 8

```

Рисунок 3.4 – Результаты заданий 2.1 и 2.2

Рекурсивный метод дает 120 для n равного 5 и 5040 для n равного 7, что совпадает с результатом итеративного метода. Метод максимума возвращает 9, 10 и 8 для наборов (3, 9, 5), (10, 2, 7) и (1, 4, 8).

3.6 Задание 3. Создание анализатора кода

Для пунктов 2.1 и 2.2 созданы классы-анализаторы *BenchmarkDotNet*. Анализатор факториала сравнивает рекурсивный метод на C#, динамический метод с циклом и динамический рекурсивный метод. Анализатор максимума сравнивает метод на C# и динамический метод. Входные числа хранятся в полях класса, чтобы компилятор не вычислил результат заранее. Код анализатора факториала приведен в листинге.

Листинг 3.5 – Анализатор факториала

```

[MemoryDiagnoser]
[ShortRunJob]
public class FactorialAnalyzer
{
    private Func<int, int> _dynamicFactorial = null!;
    private Func<int, int> _dynamicRecursiveFactorial = null!;
    private int _n = 10;

    [GlobalSetup]
    public void Setup()
    {
        _dynamicFactorial = ILGeneratorDemo.CreateFactorialMethod();
        _dynamicRecursiveFactorial = ILGeneratorDemo.CreateRecursiveFactorialMethod();
    }

    [Benchmark(Baseline = true)]
    public int CSharpRecursive() => Factorial(_n);

    [Benchmark]
    public int DynamicLoop() => _dynamicFactorial(_n);
}

```

```
[Benchmark]
public int DynamicRecursive() => _dynamicRecursiveFactorial(_n);

private static int Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);
}
```

Результаты запуска анализаторов в режиме *Release* показаны на рисунке 3.5.

```
> dotnet run -c Release -- bench
BenchmarkDotNet v0.14.0, Windows 11 (10.0.26200.9550)
.NET SDK 9.0.318
| Method | Mean | Error | StdDev | Ratio | RatioSD | Allocated | Alloc Ratio |
|-----|-----|-----|-----|-----|-----|-----|-----|
| CSharpRecursive | 12.68 ns | 49.618 ns | 2.720 ns | 1.03 | 0.26 | - | NA |
| DynamicLoop | 14.71 ns | 3.184 ns | 0.175 ns | 1.19 | 0.20 | - | NA |
| DynamicRecursive | 30.10 ns | 51.626 ns | 2.830 ns | 2.44 | 0.45 | - | NA |
BenchmarkDotNet v0.14.0, Windows 11 (10.0.26200.9550)
.NET SDK 9.0.318
| Method | Mean | Error | StdDev | Median | Ratio | RatioSD | Allocated | Alloc Ratio |
|-----|-----|-----|-----|-----|-----|-----|-----|
| CSharpMax | 0.4312 ns | 6.5541 ns | 0.3593 ns | 0.2270 ns | 1.45 | 1.36 | - | NA |
| DynamicMax | 2.4587 ns | 14.3018 ns | 0.7839 ns | 2.7281 ns | 8.29 | 4.76 | - | NA |
```

Рисунок 3.5 – Результаты *BenchmarkDotNet* для заданий 3.1 и 3.2

Для факториала от 10 среднее время составило около 12,7 нс для метода на *C#*, 14,7 нс для динамического цикла и 30,1 нс для динамического рекурсивного метода. Для максимума из трех чисел динамический метод выполняется примерно за 2,5 нс, метод на *C#* – менее чем за 1 нс. Динамические методы вызываются через делегат, что добавляет накладные расходы. Выделения памяти нет ни в одном варианте. Погрешности измерений (столбец *Error*) велики, так как использована короткая конфигурация запуска.

3.7 Исследование JIT-компиляции

Пример из методических указаний измеряет время вызова методов *NormalMethod*, *VirtualMethod* и *DelegateMethod* класса *JitBenchmark* через рефлекссию после прогрева *JIT*, затем выводит сведения о методе *Calculator.Add* и профиль трех способов вызова на 10 миллионов итераций. Результат измерения и инспекции показан на рисунке 3.6.

```

> dotnet run
=== JIT Бенчмарк (упрощенная версия) ===

Тестирование: NormalMethod
Время: 56 мс
Тики: 567,220

Тестирование: VirtualMethod
Время: 85 мс
Тики: 858,511

Тестирование: DelegateMethod
Время: 78 мс
Тики: 788,266

=== Инспекция метода: Add ===

```

Рисунок 3.6 – Измерение *JIT*-бенчмарка и инспекция метода *Add*

Обычный метод выполняется быстрее виртуального метода и вызова через делегат (56, 85 и 78 мс за 10 000 вызовов). Метод *Add* объявлен виртуальным, его *IL* занимает 9 байт.

Продолжение вывода программы, в котором определяется режим компиляции, показано на рисунке 3.7.

```

Адрес метода: 0x00007FF84CED3B58
Размер IL: 9 байт
Локальные переменные: 1
Максимальный стек: 2

Атрибуты метода:
- IsVirtual: True
- IsStatic: False
- IsAbstract: False
- IsFinal: False

=== Сравнение режимов JIT ===

TieredCompilation: ВКЛЮЧЕН

Для изменения режимов добавьте в .csproj:
<TieredCompilation>>false</TieredCompilation>

```

Рисунок 3.7 – Режим *JIT*-компиляции (продолжение вывода)

Режим *Tiered Compilation* включен по умолчанию, поэтому методы сначала компилируются быстро, а затем при частых вызовах оптимизируются повторно.

3.8 Задание 4.1. Влияние JIT-компиляции на сложение элементов массива

Метод *SumArray* суммирует 10000 целых чисел массива. Первый вызов включает JIT-компиляцию метода, последующие вызовы выполняют уже скомпилированный код. Для сравнения время первого вызова сопоставляется со средним временем 1000 повторных вызовов. Код задания приведен в листинге.

Листинг 3.6 – Задание 4.1

```
// Задание 4.1: сложение элементов массива из 10000 целых чисел
// Первый вызов включает JIT-компиляцию метода, последующие выполняют уже
// скомпилированный код
static void ArraySumTask()
{
    Console.WriteLine("\n=== Задание 4.1: сумма элементов массива (10000 элементов)
    ===\n");

    int[] array = new int[10000];
    for (int i = 0; i < array.Length; i++)
        array[i] = i;

    // Первый вызов: JIT-компиляция + выполнение
    long start = Stopwatch.GetTimestamp();
    long result = SumArray(array);
    long firstCall = Stopwatch.GetTimestamp() - start;
    Console.WriteLine($"Сумма = {result}");
    Console.WriteLine($"Первый вызов (с JIT-компиляцией): {firstCall * 1_000_000.0 /
    Stopwatch.Frequency:F1} мкс");

    // Повторные вызовы: метод уже скомпилирован
    const int calls = 1000;
    start = Stopwatch.GetTimestamp();
    for (int i = 0; i < calls; i++)
        SumArray(array);
    long repeated = Stopwatch.GetTimestamp() - start;
    Console.WriteLine($"Повторный вызов (без JIT-компиляции): {repeated * 1_000_000.0 /
    Stopwatch.Frequency / calls:F1} мкс");
}

[MethodImpl(MethodImplOptions.NoInlining)]
static long SumArray(int[] array)
{
    long sum = 0;
    for (int i = 0; i < array.Length; i++)
        sum += array[i];
    return sum;
}
```

Результаты показаны на рисунке 3.8.

```
> dotnet run
=== Профилирование производительности ===

Прямой вызов: 48 мс
Через делегат: 86 мс
Через виртуальный метод: 102 мс

=== Задание 4.1: сумма элементов массива (10000 элементов) ===

Сумма = 49995000
Первый вызов (с JIT-компиляцией): 147.3 мкс
Повторный вызов (без JIT-компиляции): 40.1 мкс
```

Рисунок 3.8 – Результаты профилирования и задания 4.1

Прямой цикл выполнился за 48 мс, вызов через делегат – за 86 мс, через виртуальный метод – за 102 мс. Сумма элементов массива равна 49995000. Первый вызов *SumArray* занял 147,3 мкс, повторные вызовы в среднем 40,1 мкс, то есть *JIT*-компиляция удлинила первый вызов примерно в 3,7 раза.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Что такое *IL*-код и зачем он нужен

IL-код – промежуточный байт-код платформы *.NET*, в который компилируются все языки платформы. Он не зависит от процессора и операционной системы, благодаря чему одна сборка выполняется на разных платформах после *JIT*-компиляции. В *IL* существуют инструкции загрузки и сохранения (*ldarg*, *ldloc*, *stloc*, *ldc*, *ldstr*), арифметики (*add*, *sub*, *mul*), переходов (*br*, *bgt*, *blt*), вызовов (*call*, *callvirt*, *ret*) и работы с объектами (*newobj*, *ldfld*, *stfld*). Команда *stloc* сохраняет значение с вершины стека в локальную переменную, а *starg* – в аргумент метода.

4.2 Разница между *Normal JIT* и *Tiered Compilation*

При *Normal JIT* метод компилируется один раз при первом вызове с полной оптимизацией. При *Tiered Compilation* метод сначала компилируется быстро и без глубокой оптимизации, а затем часто вызываемые методы перекомпилируются в оптимизированный вариант. Это ускоряет запуск программы и сохраняет высокую скорость горячих участков.

4.3 Холодный старт и способы его уменьшения

Холодный старт – это медленное выполнение первых вызовов из-за *JIT*-компиляции методов и загрузки сборок. Его уменьшают предварительной компиляцией *ReadyToRun* или *Native AOT*, быстрой начальной компиляцией (*Tiered Compilation*), прогревом методов перед ответственным участком.

4.4 Оптимизация часто вызываемых методов

Среда выполнения считает вызовы методов. Метод, превысивший порог, перекомпилируется в более высокий уровень с оптимизациями: встраиванием коротких методов, оптимизацией циклов и использованием профиля выполнения.

4.5 Преимущества генерации кода через *ILGenerator*

Генерация позволяет создавать методы во время выполнения без исходного кода, подстраивать логику под данные и получать скорость, близкую к обычному коду, если делегат создан один раз и затем

переиспользуется. Динамическая компиляция применяется в *ORM* и сериализаторах, *DI*-контейнерах, библиотеках прокси и мокирования, движках скриптов и вычислителях выражений.

ЗАКЛЮЧЕНИЕ

Изучены структура *IL* и основные команды: получены и разобраны байты *IL* методов сложения, умножения, цикла и условия. Создан набор динамических методов через *DynamicMethod* и *ILGenerator*, включая рекурсивный факториал и максимум из трех чисел. Анализаторы *BenchmarkDotNet* показали, что динамические методы, вызываемые через делегат, уступают обычному коду на единицы наносекунд. Исследование JIT-компиляции показало, что первый вызов метода дольше последующих примерно в 3,7 раза из-за затрат на компиляцию. Цель работы достигнута.